

Data Structures and Algorithms — Model Answers

Question Bank (Units I & II) — S.Y. B.Tech, Semester III, A.Y. 2026-27

Each answer is prepared for 5 marks, with theory, algorithm/pseudo-code, diagrams and examples as applicable.

Unit I: Introduction to Data Structures and Algorithm Analysis

Q1. Define data, information, and knowledge with suitable examples. Also explain the role of data structures.

5M

Definitions

- **Data** — Raw, unprocessed facts and figures that have no meaning by themselves. Example: 90, 85, 78 (marks of a student in three subjects).
- **Information** — Data that has been processed, organised or interpreted to give it meaning. Example: "The student scored an average of 84.3%."
- **Knowledge** — The understanding gained by combining information with experience/context, used for decision making. Example: "Since the average is above 80%, the student qualifies for a merit scholarship."

Role of Data Structures

A data structure is a systematic way of organising, storing and managing data so that it can be used efficiently. Data structures help to:

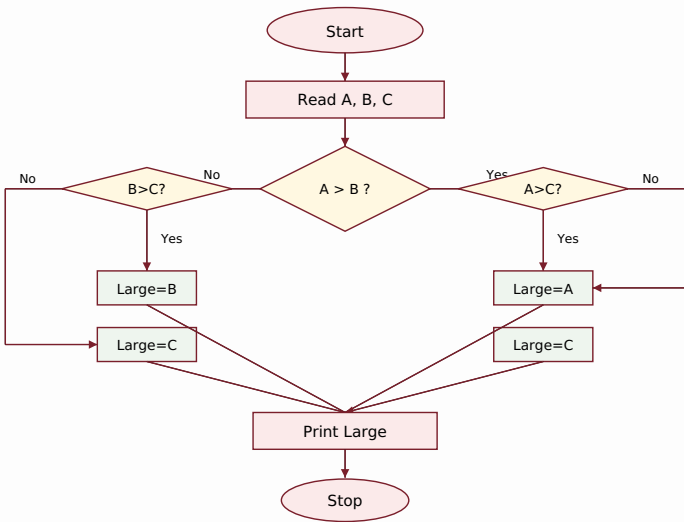
- Store large volumes of data in an organised manner.
- Perform operations such as insertion, deletion, searching and sorting efficiently.
- Reduce time and space complexity of algorithms.
- Serve as the backbone of efficient software, databases, and operating systems.

Q2. Design an algorithm (pseudo-code & flowchart) to find the largest of three numbers; compare the two representations. 5M

Pseudo-code

```
Algorithm Largest(A, B, C)
Step 1: Start
Step 2: Read A, B, C
Step 3: if (A > B) then
    if (A > C) then
        Large = A
    else
        Large = C
else
    if (B > C) then
        Large = B
    else
        Large = C
Step 4: Print Large
Step 5: Stop
```

Flowchart



Comparison of the two representations

Pseudo-code	Flowchart
Text-based, close to a programming language	Graphical/symbolic, uses standard shapes (oval, rectangle, diamond)
Easier to convert directly into code	Easier to visualise the logic and control flow
Compact, good for complex logic	Can become large/unwieldy for complex logic

Q3. What is an Abstract Data Type (ADT)? Explain with a suitable example. 5M

An **Abstract Data Type** is a mathematical/logical model of a data structure that defines **what** operations can be performed on the data, without specifying **how** those operations are implemented internally. It separates the interface (operations visible to the user) from the implementation (internal representation), a concept known as **data abstraction / encapsulation**.

Example: Stack ADT

Data: Collection of elements arranged in a particular order.

Operations: push(x) – insert element, pop() – remove top element, peek() – view top element, isEmpty() – check if stack is empty.

The user only knows *what* push/pop do (LIFO behaviour), while the internal implementation may use an **array** or a **linked list** — the user does not need to know this detail.

Other common ADT examples: Queue, List, Tree, Graph.

Q4. Compare and justify, with real-world examples, linear vs non-linear and static vs dynamic data structures.

5M

Linear Data Structure	Non-Linear Data Structure
Elements are arranged in a sequential order, one after another.	Elements are arranged in a hierarchical or interconnected manner.
Each element has a unique predecessor and successor (except first/last).	An element may be connected to multiple elements.
Examples: Array, Stack, Queue, Linked List	Examples: Tree, Graph
Real world: A queue of people at a bank counter (one behind another).	Real world: An organisation chart where one manager supervises many employees (tree), or a road network connecting cities (graph).

Static Data Structure	Dynamic Data Structure
Size is fixed at compile time and cannot change during execution.	Size can grow or shrink at run time as required.
Memory is allocated once, in a contiguous block.	Memory is allocated/de-allocated dynamically, often non-contiguously.
Example: Array	Example: Linked List
Real world: Fixed number of seats in a train coach.	Real world: A queue of customers in a shop, which keeps growing/shrinking as people join or leave.

Q5. Differentiate between static and dynamic data structures with suitable examples.

5M

Point	Static	Dynamic
Memory allocation	Fixed, at compile time	Variable, at run time
Memory location	Contiguous	May be scattered (linked via pointers)
Size modification	Not possible once declared	Possible — insert/delete as needed
Memory utilisation	May waste memory if not fully used	Efficient — allocates only as much as needed
Example	Array: <code>int arr[10];</code>	Linked List, using <code>malloc()/new</code>
Access speed	Fast (direct/random access)	Slower (sequential access via pointers)

Q6. Define an algorithm. State its essential characteristics and explain with a simple example.

5M

An **algorithm** is a finite, well-defined sequence of unambiguous steps that takes some input, processes it, and produces the desired output to solve a specific problem.

Essential Characteristics

- **Input** — zero or more well-defined inputs.
- **Output** — at least one well-defined output.
- **Finiteness** — must terminate after a finite number of steps.
- **Definiteness** — each step must be clear and unambiguous.
- **Effectiveness** — each step must be simple enough to be carried out.
- **Feasibility** — must be achievable with available resources.

Example — Sum of two numbers

```
Step 1: Start
Step 2: Read A, B
Step 3: Sum = A + B
Step 4: Print Sum
Step 5: Stop
```

Q7. What is algorithm complexity analysis? Explain time and space complexity with suitable examples.

5M

Complexity analysis is the process of determining the amount of computing resources (time and memory) an algorithm needs, as a function of the input size *n*. It helps compare efficiency of different algorithms independent of hardware/language.

Time Complexity

The amount of time an algorithm takes to run, expressed as a function of input size *n*. Example: For linear search on *n* elements, in the worst case every element is compared once, so time complexity = $O(n)$.

Space Complexity

The total amount of memory an algorithm needs, including space for input, variables and auxiliary data structures. Space Complexity = Fixed part (constants, simple variables) + Variable part (dynamic memory, recursion stack).

Example: To sum *n* numbers stored in an array, we need $O(n)$ space for the array and $O(1)$ extra space for the sum variable → overall space complexity = $O(n)$.

Q8. Explain the Step Count Method of analyzing time complexity with a suitable example.

5M

The **Step Count Method** estimates time complexity by counting how many times each statement executes, assigning a step count to every line, and summing the total steps as a function of input size n .

Example: Sum of elements of an array

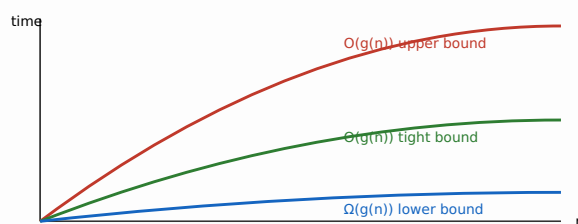
	steps/exec	frequency	total
1 int sum(int A[], int n){	0	-	0
2 int s = 0;	1	1	1
3 for(i = 0; i < n; i++)	1	(n+1)	n+1
4 s = s + A[i];	1	n	n
5 return s;	1	1	1
6 }	0	-	0

Total steps = $1 + (n+1) + n + 1 = 2n + 3$. Ignoring constants, this gives **$O(n)$** — a linear time algorithm.

Q9. Explain the asymptotic notations Big-O, Theta and Omega with suitable examples.

5M

- **Big-O (O) — Upper bound:** Represents the worst-case running time; $f(n) = O(g(n))$ if $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Example: Linear search worst case = $O(n)$.
- **Omega (Ω) — Lower bound:** Represents the best-case running time; $f(n) = \Omega(g(n))$ if $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$. Example: Linear search best case = $\Omega(1)$ (key found at first position).
- **Theta (Θ) — Tight bound:** Represents the average/exact growth rate when upper and lower bounds match; $f(n) = \Theta(g(n))$ if $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$. Example: Bubble sort in all cases (ignoring best-case optimisation) is $\Theta(n^2)$.



Q10. Explain best case, worst case, and average case time complexity with a suitable example.

5M

- **Best Case:** The minimum time taken by the algorithm for any input of size n (most favourable input).
- **Worst Case:** The maximum time taken by the algorithm for any input of size n (least favourable input).
- **Average Case:** The expected time taken over all possible inputs of size n .

Example — Linear Search for key K in array of n elements:

- Best case: K is the first element $\rightarrow O(1)$
- Worst case: K is the last element or not present $\rightarrow O(n)$
- Average case: K is found, on average, near the middle $\rightarrow O(n)$

Q11. Explain the concept of sequential organization in data storage with a suitable example.

5M

Sequential organisation stores data elements in consecutive (contiguous) memory locations, one after another, in the order they are inserted or in a specific order (e.g. sorted). The address of the next element can be computed directly from the address of the first element, allowing fast direct/random access.

0	1	2	3	4
12	25	7	31	9

Example: An array of 5 integers stored starting at address 1000 (each 4 bytes): element at index i is located at $1000 + i \times 4$. This is the basis of arrays, sequential files, and contiguous memory allocation.

Q12. Describe how arrays serve as an Abstract Data Type (ADT) with suitable examples.

5M

As an ADT, an **Array** is defined by the operations it supports on a collection of same-type elements stored under a common name, rather than by how it is implemented in memory.

Operations defined by the Array ADT

- **Create(size)** — creates an array of given size.
- **Insert(A, i, x)** — inserts element x at index i .
- **Delete(A, i)** — removes the element at index i .
- **Traverse(A)** — visits every element (e.g. to print/process).
- **Search(A, x)** — finds the position of x .
- **Update(A, i, x)** — modifies value at index i .

Example: An array `Marks[5]` supports Traverse (print all marks), Search (find a specific score), and Update (correct a score) without the user needing to know that elements are stored contiguously in memory.

Q13. A 2D array A[4][5] is row-major with base address 1000, element size 4 bytes. Find the address of A[2][3]. How does it differ in column-major order? 5M

Given

Rows (R) = 4, Columns (C) = 5, Base Address = 1000, Size (W) = 4 bytes, element = A[2][3] (0-indexed: i=2, j=3)

Row-Major Order

```
Address(A[i][j]) = Base + [(i × C) + j] × W
                 = 1000 + [(2 × 5) + 3] × 4
                 = 1000 + [10 + 3] × 4
                 = 1000 + 13 × 4
                 = 1000 + 52
                 = 1052
```

Column-Major Order

```
Address(A[i][j]) = Base + [(j × R) + i] × W
                 = 1000 + [(3 × 4) + 2] × 4
                 = 1000 + [12 + 2] × 4
                 = 1000 + 14 × 4
                 = 1000 + 56
                 = 1056
```

Conclusion: In row-major order elements of a row are stored contiguously (address = 1052), while in column-major order elements of a column are stored contiguously (address = 1056). The two orders give different physical addresses for the same logical element because the traversal order of the elements differs.

Q14. Define an Ordered List. Explain its properties, advantages, and basic operations with examples. 5M

An **Ordered List** (sorted list) is a linear list in which elements are arranged according to some order — ascending or descending — based on a key value, and this order is maintained even after insertions/deletions.

Properties

- Elements follow a defined order (e.g. numeric or alphabetic).
- New elements are inserted at the correct position to preserve order.
- Can be implemented using arrays or linked lists.

Advantages

- Searching can use efficient methods like Binary Search — $O(\log n)$.
- Finding minimum/maximum is $O(1)$ (first/last element).
- Useful for merging, ranking, and range queries.

Basic Operations

- **Insert(x)** — insert x while maintaining sorted order.
- **Delete(x)** — remove x, list remains sorted.
- **Search(x)** — locate x (e.g. via binary search).
- **Traverse()** — display all elements in order.

Example: Ordered list [5, 12, 19, 27, 33]. Inserting 20 gives [5, 12, 19, 20, 27, 33] — the element is placed between 19 and 27 to preserve order.

Q15. Explain how a polynomial can be represented using an array. Illustrate with an example. 5M

A polynomial such as $P(x) = c_nx^n + c_{n-1}x^{n-1} + \dots + c_1x + c_0$ can be represented using a 1-D array where the **index represents the exponent of x** and the **value stored represents the coefficient**.

Example: $P(x) = 6x^4 + 0x^3 + 3x^2 + 5x + 7$

Exp: 0 1 2 3 4

7	5	3	0	6
---	---	---	---	---

An alternative representation uses **parallel arrays** (or an array of (coefficient, exponent) pairs) storing only the non-zero terms — useful for sparse polynomials, e.g. $\text{Coeff}[] = \{6,3,5,7\}$, $\text{Exp}[] = \{4,2,1,0\}$. This saves memory when most coefficients are zero. Arrays are convenient here because polynomial addition/multiplication reduces to simple index-wise operations.

Q1. Explain the classification of sorting as internal and external sorting. Also define sort stability with examples.

5M

Internal Sorting	External Sorting
All data to be sorted fits entirely in main memory (RAM).	Data is too large to fit in RAM; stored on secondary storage (disk).
Faster, since RAM access is quick.	Slower, needs multiple read/write passes over disk.
Examples: Bubble sort, Selection sort, Insertion sort, Quick sort, Heap sort.	Example: External Merge Sort.

Sort Stability

A sorting algorithm is **stable** if it preserves the relative order of records with **equal keys** as they appeared in the original input.

Example: Records (Name, Marks) = (A,50), (B,30), (C,50). After a stable sort by Marks: (B,30), (A,50), (C,50) — A still appears before C, since A came before C originally. An unstable sort might output (B,30), (C,50), (A,50).

Q2. Explain the Linear Search algorithm. Write pseudo-code, analyze time complexity, and give an example.

5M

Linear Search checks every element of the array sequentially, one by one, from the beginning until the desired key is found or the array ends.

```
Algorithm LinearSearch(A, n, key)
Step 1: for i = 0 to n-1
Step 2:     if A[i] == key then
              return i          // found at index i
Step 3: return -1                // not found
```

Time Complexity

Best case: O(1) (key at first position). Worst/Average case: O(n) (key at last position or absent). Space complexity: O(1).

Example: A = [4, 9, 2, 7, 15], key = 7. Comparisons: 4≠7, 9≠7, 2≠7, 7=7 → found at index 3.

Q3. Apply the Sentinel Search technique to search for a key. Write the algorithm; analyze how it reduces comparisons vs Linear Search.

5M

Sentinel Search is an optimisation of linear search: the key is temporarily placed at the end of the array (as a "sentinel"), removing the need for a separate boundary check (i < n) in every loop iteration — only the key-match check is needed.

```
Algorithm SentinelSearch(A, n, key)
Step 1: last = A[n-1]           // save last element
Step 2: A[n-1] = key           // place sentinel
Step 3: i = 0
Step 4: while A[i] != key
          i = i + 1
Step 5: A[n-1] = last           // restore last element
Step 6: if (i < n-1) or (last == key)
          return i (or n-1)    // found
        else
          return -1            // not found
```

Example: A = [4, 9, 2, 7, 15], key = 7. Sentinel placed: A=[4,9,2,7,7]. Loop compares only equality (no bound check), stops at i=3.

Reduction in comparisons

Ordinary linear search performs **2 comparisons per iteration** (i < n and A[i] == key). Sentinel search performs only **1 comparison per iteration** (A[i] == key), since the sentinel guarantees the loop will always terminate. This nearly **halves** the number of comparisons, improving practical performance while keeping worst-case complexity O(n).

Q4. Explain the Binary Search algorithm with a simple example.

5M

Binary Search works only on a **sorted** array. It repeatedly compares the key with the middle element and eliminates half of the search space at each step.

```
Algorithm BinarySearch(A, n, key)
Step 1: low = 0, high = n-1
Step 2: while low <= high
    mid = (low + high) / 2
    if A[mid] == key
        return mid
    else if A[mid] < key
        low = mid + 1
    else
        high = mid - 1
Step 3: return -1 // not found
```

Example: A = [3, 8, 12, 19, 27, 34, 41], key = 27
low=0, high=6, mid=3 → A[3]=19 < 27 → low=4
low=4, high=6, mid=5 → A[5]=34 > 27 → high=4
low=4, high=4, mid=4 → A[4]=27 = key → **found at index 4**

Time complexity: $O(\log n)$ in worst/average case, $O(1)$ in best case; Space complexity: $O(1)$ iterative / $O(\log n)$ recursive (stack).

Q5. Explain the Bubble Sort algorithm with a suitable example. State whether it is stable.

5M

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest unsorted element "bubbles up" to its correct position at the end.

```
Algorithm BubbleSort(A, n)
Step 1: for i = 0 to n-2
Step 2:     for j = 0 to n-2-i
Step 3:         if A[j] > A[j+1]
                swap(A[j], A[j+1])
```

Example: A = [5, 1, 4, 2]
Pass1: [1,5,4,2] → [1,4,5,2] → [1,4,2,5]
Pass2: [1,4,2,5] → [1,2,4,5]
Pass3: [1,2,4,5] (no swaps) → Sorted: [1,2,4,5]

Time complexity: $O(n^2)$ worst/average, $O(n)$ best (with swap-flag optimisation). Space complexity: $O(1)$.

Stability: Bubble Sort is **stable**, because it swaps only when a strictly greater element precedes a smaller one — equal elements are never swapped, so their relative order is preserved.

Q6. Explain the adaptive nature of Bubble Sort and discuss its stability, with an illustration for already-sorted data.

5M

An algorithm is **adaptive** if it performs better (fewer operations) when the input is already partially or fully sorted. Bubble Sort becomes adaptive when implemented with a **swapped flag**: if a complete pass makes no swaps, the array is already sorted, and the algorithm terminates early.

```
Algorithm AdaptiveBubbleSort(A, n)
Step 1: for i = 0 to n-2
Step 2:     swapped = false
Step 3:     for j = 0 to n-2-i
Step 4:         if A[j] > A[j+1]
                swap(A[j], A[j+1]); swapped = true
Step 5:     if swapped == false
                break // already sorted, stop early
```

Example (already sorted): A = [1, 2, 3, 4, 5]. In pass 1, every comparison $A[j] > A[j+1]$ is false, so swapped stays false → the algorithm exits after just **one pass** (**$O(n)$**) instead of the usual $O(n^2)$, demonstrating adaptiveness.

Stability: As in Q5, Bubble Sort remains stable in the adaptive version too, since equal-valued elements are never swapped past each other — the swap condition uses strict ">", not "≥".

Q7. Explain the Selection Sort algorithm. Write pseudo-code/C++ and discuss procedure, complexity, and stability.

5M

Selection Sort divides the array into a sorted and an unsorted part. In each pass, it finds the **minimum** element from the unsorted part and swaps it with the first element of the unsorted part.

```
Algorithm SelectionSort(A, n)
Step 1: for i = 0 to n-2
Step 2:     min_idx = i
Step 3:     for j = i+1 to n-1
Step 4:         if A[j] < A[min_idx]
                min_idx = j
Step 5:     swap(A[i], A[min_idx])
```

Example: A = [29, 10, 14, 37, 13] → after pass1 (min=10): [10,29,14,37,13] → pass2 (min=13): [10,13,14,37,29] → pass3: [10,13,14,37,29] → pass4 (min=29): [10,13,14,29,37] → Sorted.

Time complexity: $O(n^2)$ in best, average, and worst case (always scans remaining elements). **Space complexity:** $O(1)$.

Stability: Selection Sort is generally **unstable**, because swapping the minimum element with the first unsorted position can move an element past an equal-valued element, changing their relative order.

Q8. Write a C++ program for Selection Sort. Comment on its time complexity.

5M

```
#include <iostream>
using namespace std;

void selectionSort(int A[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (A[j] < A[min_idx])
                min_idx = j;
        }
        swap(A[i], A[min_idx]);
    }
}

int main() {
    int A[] = {29, 10, 14, 37, 13};
    int n = sizeof(A) / sizeof(A[0]);
    selectionSort(A, n);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << A[i] << " ";
    return 0;
}
// Output: Sorted array: 10 13 14 29 37
```

Time complexity comment: The outer loop runs $(n-1)$ times and the inner loop runs $(n-i-1)$ times on average, giving total comparisons $\approx n(n-1)/2$, so the time complexity is **$O(n^2)$** in the best, average and worst case — it does not improve for already-sorted input, since it always scans the remaining unsorted part to find the minimum. Space complexity is $O(1)$ (in-place, no extra array).

Q9. Explain the Insertion Sort algorithm. Write pseudo-code/C++, discuss procedure, complexity, stability and adaptiveness.

5M

Insertion Sort builds the sorted array one element at a time — it picks each element from the unsorted part and inserts it into its correct position within the already-sorted part, similar to how one sorts playing cards in hand.

```
Algorithm InsertionSort(A, n)
Step 1: for i = 1 to n-1
Step 2:     key = A[i]
Step 3:     j = i - 1
Step 4:     while j >= 0 and A[j] > key
                A[j+1] = A[j]
                j = j - 1
Step 5:     A[j+1] = key
```

Example: A = [12, 11, 13, 5, 6] → insert 11: [11,12,13,5,6] → insert 13: [11,12,13,5,6] → insert 5: [5,11,12,13,6] → insert 6: [5,6,11,12,13] Sorted.

Time complexity: Best case $O(n)$ (already sorted — inner while never runs); Worst/Average case $O(n^2)$ (reverse sorted). **Space complexity:** $O(1)$.

Stability: Insertion Sort is **stable** — it shifts elements only while they are strictly greater than the key, so equal elements retain their original order.

Adaptiveness: It is **adaptive** — the number of shifts depends directly on how unsorted the input is; for nearly-sorted data it runs close to $O(n)$.

Q10. Define a Heap. Explain the Heap data structure, its types, properties, and basic operations.

5M

A **Heap** is a complete binary tree that satisfies the **heap property**: the key at each node is ordered with respect to its children's keys. It is commonly implemented using an array, where for a node at index i : left child = $2i+1$, right child = $2i+2$, parent = $(i-1)/2$.

Types

- **Max Heap**: Value of every parent node $\geq$ values of its children; the root holds the maximum element.
- **Min Heap**: Value of every parent node $\leq$ values of its children; the root holds the minimum element.

Properties

- Complete binary tree — all levels filled except possibly the last, which fills left to right.
- Height of a heap with n nodes is $O(\log n)$.
- Efficiently represented using an array (no pointers needed).

Basic Operations

- **Insert(x)** — add x at the next free position, then "heapify-up" (bubble up) to restore heap property — $O(\log n)$.
- **Delete-Max/Min()** — remove the root, move the last element to root, then "heapify-down" (sift down) — $O(\log n)$.
- **Peek()** — return root element (max/min) — $O(1)$.
- **Build-Heap(A)** — convert an arbitrary array into a heap — $O(n)$.

Q11. What is a Min Heap and Max Heap? Sort 77, 15, 91, 21, 6, 46 using Heap Sort in ascending order.

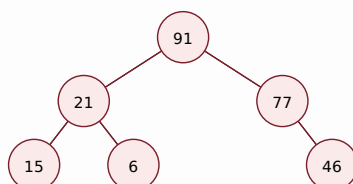
5M

Min Heap: Every parent's value is $\leq$ its children's values (smallest element at root). **Max Heap**: Every parent's value is $\geq$ its children's values (largest element at root). Heap Sort for **ascending** order uses a **Max Heap**: repeatedly extract the maximum (root) and place it at the end of the array.

Step 1: Build Max Heap from [77, 15, 91, 21, 6, 46]

Starting from the last internal node and sifting down:

```
Initial:          [77, 15, 91, 21, 6, 46]
Heapify idx1(15): children 21,6 → swap 15,21 → [77, 21, 91, 15, 6, 46]
Heapify idx0(77): children 21,91 → swap 77,91 → [91, 21, 77, 15, 6, 46]
Max Heap built:   [91, 21, 77, 15, 6, 46]
```



Step 2: Repeated extraction (swap root with last, sift down, shrink heap)

```
[91,21,77,15,6,46] → swap(91,46) → [46,21,77,15,6 | 91] → sift → [77,21,46,15,6 | 91]
[77,21,46,15,6] → swap(77,6) → [6,21,46,15 | 77,91] → sift → [46,21,6,15 | 77,91]
[46,21,6,15] → swap(46,15) → [15,21,6 | 46,77,91] → sift → [21,15,6 | 46,77,91]
[21,15,6] → swap(21,6) → [6,15 | 21,46,77,91] → sift → [15,6 | 21,46,77,91]
[15,6] → swap(15,6) → [6 | 15,21,46,77,91]
```

Final Sorted (ascending): 6, 15, 21, 46, 77, 91

Result: Sorted array = **6, 15, 21, 46, 77, 91**. Time complexity of Heap Sort: $O(n \log n)$ in all cases; Space complexity: $O(1)$ (in-place).

Q12. What is hashing? Explain the concept and role of hashing in efficient searching, with a suitable example.

5M

Hashing is a technique that maps a key to a specific location (index/slot) in a data structure called a **hash table**, using a mathematical function called a **hash function**, $h(\text{key})$. This allows data to be inserted and searched in close to **$O(1)$** average time, instead of scanning the whole collection.

Role in Efficient Searching

- Direct computation of the storage location avoids sequential/binary search overhead.
- Average-case Search, Insert, Delete are all $O(1)$, compared to $O(n)$ for linear search or $O(\log n)$ for binary search.
- Widely used in databases, compilers (symbol tables), caches, and password storage.

Example: $h(\text{key}) = \text{key} \bmod 10$, with a table of size 10. To insert key 27 $\rightarrow h(27)=7 \rightarrow$ store at slot 7. To search for 27, we simply compute $h(27)=7$ and check slot 7 directly — no scanning needed.

Q13. Hash table of size 7, keys {23, 44, 12, 18, 39, 55}. Apply division-modulo hashing; resolve collisions with linear probing; show the final table.

5M

Hash function: $h(\text{key}) = \text{key} \bmod 7$. Linear probing: on collision, check the next slot $(h(\text{key})+1, h(\text{key})+2, \dots \bmod 7)$ until an empty slot is found.

Key	$h(\text{key})=\text{key}\%7$	Action
23	2	slot 2 empty → place at 2
44	2	slot 2 full → probe 3 (empty) → place at 3
12	5	slot 5 empty → place at 5
18	4	slot 4 empty → place at 4
39	4	slot 4 full → probe 5 (full) → probe 6 (empty) → place at 6
55	6	slot 6 full → probe 0 (empty, wrap-around) → place at 0

Final Hash Table

0	1	2	3	4	5	6
55	—	23	44	18	12	39

Note how 44 collided with 23 and moved to slot 3; 39 collided with 18 (slot4) and 12 (slot5), moving to slot 6; 55 collided with 39 (slot 6) and wrapped around to slot 0 — this cascading effect is called **clustering**, a known drawback of linear probing.

Q14. Explain different types of hash functions with suitable examples.

5M

- **Division Method:** $h(\text{key}) = \text{key} \bmod m$ (m = table size, ideally a prime). Example: $\text{key}=27, m=10 \rightarrow h(27)=7$.
- **Multiplication Method:** $h(\text{key}) = \lfloor m \times (\text{key} \times A \bmod 1) \rfloor$, where $0 < A < 1$ (e.g. $A \approx 0.618$). Less sensitive to the choice of m .
- **Mid-Square Method:** Square the key, then extract the middle digits as the hash value. Example: $\text{key}=54 \rightarrow 54^2=2916 \rightarrow$ middle digits "91" $\rightarrow h=91$.
- **Folding Method:** Split the key into equal parts, add them, and use the result ($\bmod$ table size) as the hash value. Example: $\text{key}=123456$, split into 12,34,56 $\rightarrow \text{sum}=102 \rightarrow h(\text{key})=102 \bmod m$.
- **Digit/Character Analysis:** Selects specific digits/characters from the key to compute the hash — used for large alphanumeric keys.

Q15. What are the different collision resolution techniques in hashing? Explain open hashing / separate chaining with an example.

5M

Collision Resolution Techniques

- **Open Addressing (Closed Hashing):** All elements stored within the table itself. Includes Linear Probing, Quadratic Probing, and Double Hashing.
- **Separate Chaining (Open Hashing):** Each table slot holds a linked list (chain) of all keys that hash to that slot.

Separate Chaining — Explanation

Each slot of the hash table points to a linked list. When a collision occurs, the new key is simply appended to the linked list at that slot, instead of searching for another empty slot. This avoids clustering and allows the table to hold more elements than its size.

Example: Table size = 5, $h(\text{key}) = \text{key} \bmod 5$, keys inserted = 12, 17, 22, 3
 $h(12)=2, h(17)=2, h(22)=2, h(3)=3$
Slot 2 $\rightarrow 12 \rightarrow 17 \rightarrow 22$ (linked list, chained in insertion order)
Slot 3 $\rightarrow 3$
All other slots empty.

Average search time with chaining is $O(1 + \alpha)$, where α (load factor) = n/m (number of keys / table size).